SCAYLE
Supercomputación Castilla y León

Bloque 3: *Quantum Machine Learning*

Dra. Carmen Calvo-Olivera
Técnico *High Performance Computing y Quantum Computing*

SCAYLE SUPERCOMPUTACIÓN CASTILLA Y LEÓN
universidad de león
NOS IMPULSA
Junta de Castilla y León
Europa impulsa nuestro crecimiento
FONDO EUROPEO DE DESARROLLO REGIONAL
UNIÓN EUROPEA
Quantum SPAIN
GOBIERNO DE ESPAÑA
MINISTERIO DE ASUNTOS ECONÓMICOS Y TRANSFORMACIÓN DIGITAL
Plan de Recuperación, Transformación y Resiliencia
Financiado por la Unión Europea
NextGenerationEU
RES
RED ESPAÑOLA DE SUPERCOMPUTACIÓN
Infraestructuras Científicas y Técnicas Singulares
GOBIERNO DE ESPAÑA
MINISTERIO DE CIENCIA E INNOVACIÓN
Unión Europea
Fondo Europeo de Desarrollo Regional
"Una manera de hacer Europa"

ADVERTENCIA



Bloque 3 – ¿Qué veremos?

1. **Introducción al QML**
   Taxonomía, bucle híbrido, codificación de datos, kernels vs. modelos variacionales, límites actuales
2. **Casos de uso**
   Clasificación o clustering con modelos cuánticos
3. **ML vs. QML**
   Mismo dataset, misma API: referencia clásica frente a modelos cuánticos
4. **Práctica**
   Laboratorio 1 (Qiskit) y Laboratorio 2 (PennyLane) en simuladores

Machine Learning desde el punto de vista del software

Un modelo es una función parametrizada $f(x; \theta)$. Entrenar = elegir θ para minimizar una función de pérdida sobre los datos.

1 Supervisado

Datos etiquetados (x, y).
Clasificación: y discreta.
Regresión: y continua.

2 No supervisado

Solo x. Buscar estructura: clustering y reducción de dimensionalidad.

3 Semisupervisado

Pocas etiquetas + muchos ejemplos sin etiquetar.

4 Por refuerzo

Un agente aprende una política a partir de recompensas al interactuar con un entorno.

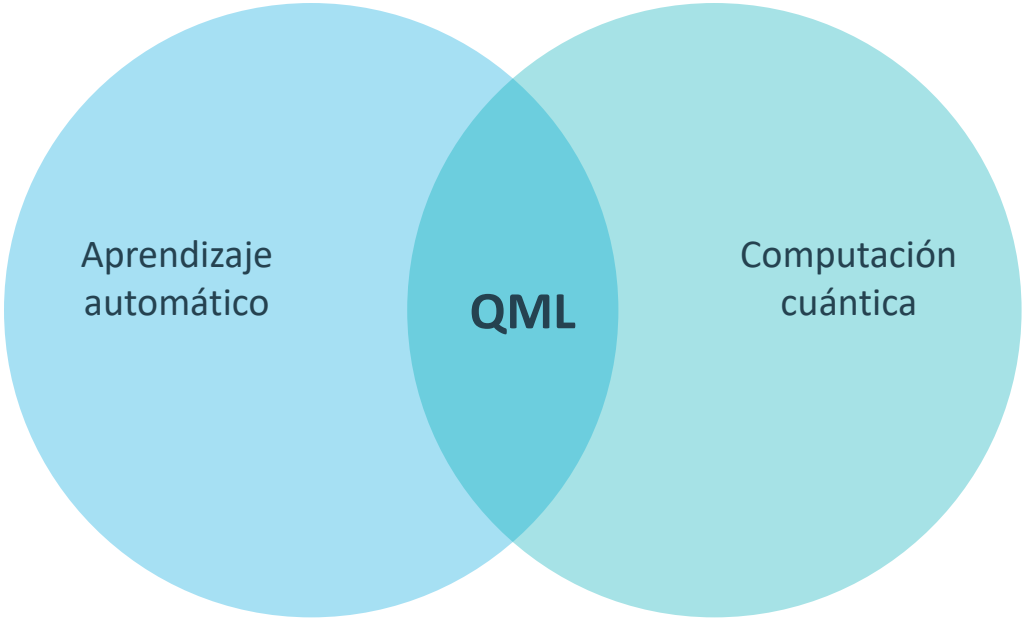
Pipeline no cambia: Seguimos preprocesando, separando entrenamiento y test, definiendo una pérdida, optimizando y evaluando**.**

En QML solo cambia el **modelo** (o el kernel): parte de $f(x; \theta)$ se evalúa en un circuito cuántico.

Refs.: [2] Schuld y Petruccione (2021); [29] Pedregosa et al. (2011).

¿Qué es el Quantum Machine Learning (QML)?

- El QML surge de combinar técnicas y algoritmos cuánticos con estrategias habituales de *machine learning*.
- La mayor parte del trabajo actual: **algoritmos híbridos cuántico-clásicos**, en los que híbrido: un circuito cuántico parametrizado calcula algo (un valor esperado o el solapamiento entre dos estados cuánticos) y un ordenador clásico se encarga de la optimización.
- Dominan dos familias de modelos: **kernels cuánticos** y **modelos cuánticos variacionales** (QNN).
- El hardware actual es NISQ: decenas o cientos de qubits pero sin corrección de errores completa; es el contexto de hardware de todo lo que hacemos hoy.



Visión de ingeniería: un circuito cuántico se convierte en un nuevo tipo de capa o de función de similitud dentro de un stack de ML clásico.

Refs.: [1] Biamonte et al., Nature (2017); [5] Cerezo et al., Nat. Rev. Phys. (2021); [25] Preskill, Quantum (2018).

Áreas de trabajo QML: datos y algoritmos

Tipo de algoritmo →

↑ *Tipo de dato*	Clásico	Cuántico
Clásico	**CC** Datos clásicos, algoritmo clásico (inspirado en cuántica)	**CQ** Datos clásicos, algoritmo cuántico ← este bloque
Cuántico	**QC** Datos cuánticos, algoritmo clásico (p. ej. aprender de experimentos)	**QQ** Datos cuánticos, algoritmo cuántico

Refs.: [3] Aïmeur, Brassard y Gambs (2006); [4] Dunjko, Taylor y Briegel, PRL (2016); [18] Huang et al. (2021).

SCAYLE SUPERCOMPUTACIÓN CASTILLA Y LEÓN | Junta de Castilla y León

Áreas de trabajo QML: datos y algoritmos

Tipo de algoritmo →

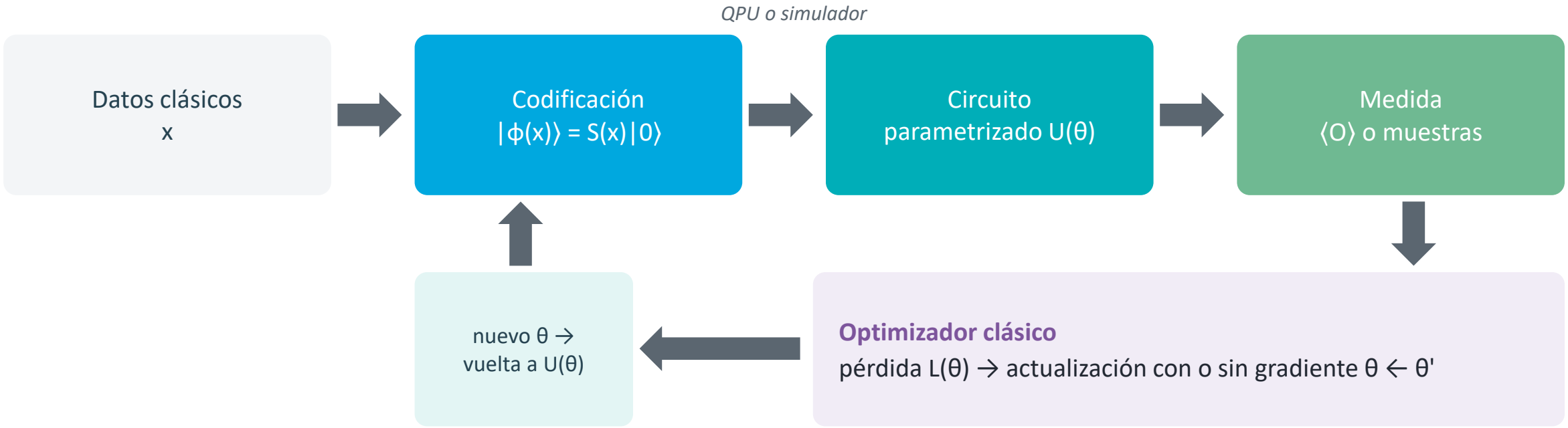
↑ *Tipo de dato*	Clásico	Cuántico
Clásico	**CC** Datos clásicos, algoritmo clásico (inspirado en cuántica)	**CQ** Datos clásicos, algoritmo cuántico ← este bloque
Cuántico	**QC** Datos cuánticos, algoritmo clásico (p. ej. aprender de experimentos)	**QQ** Datos cuánticos, algoritmo cuántico

Por qué nos importa CQ

- Nuestros datos (imágenes, logs, métricas de trabajos HPC, datos tabulares) son clásicos.
- Hay que codificarlos en estados cuánticos antes de que un circuito pueda procesarlos.
- En esa codificación aparecen muchos cuellos de botella prácticos (coste, expresividad, entrenabilidad).
- QQ es donde se esperan las ventajas teóricas más claras, porque el dato ya es cuántico.

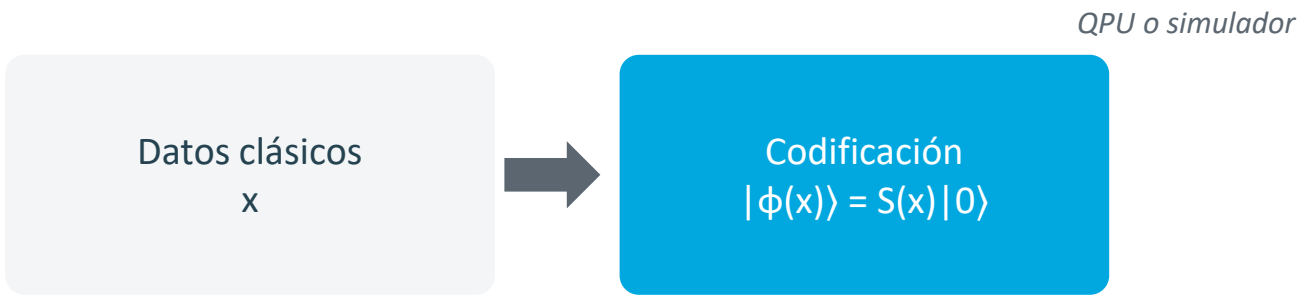
Refs.: [3] Aïmeur, Brassard y Gambs (2006); [4] Dunjko, Taylor y Briegel, PRL (2016); [18] Huang et al. (2021).

El bucle híbrido cuántico-clásico



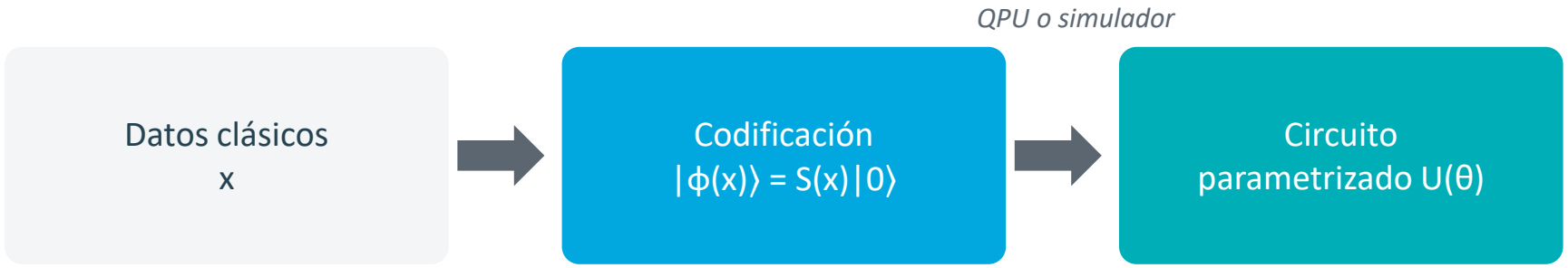
Refs.: [5] Cerezo et al. (2021); [6] Benedetti et al. (2019); [13] Mitarai et al. (2018); [28] Macaluso et al. (2020).

El bucle híbrido cuántico-clásico



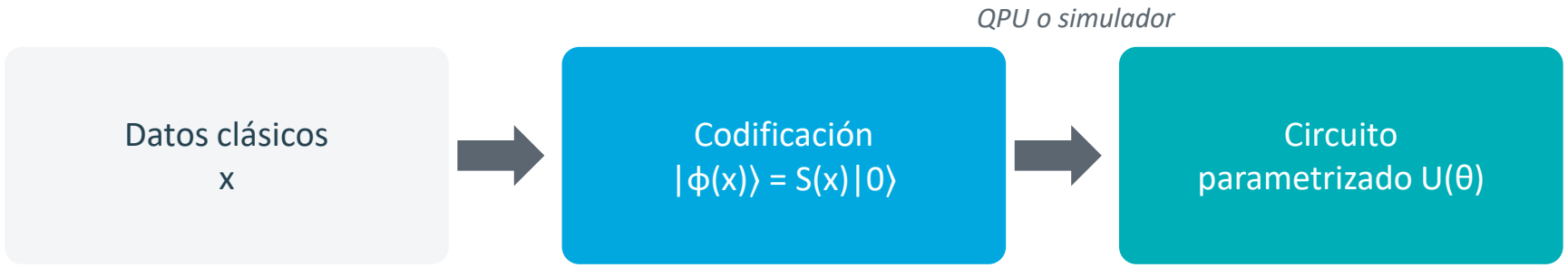
Paso 1, preparación del estado o feature encoding: un circuito S(x) traslada el vector clásico x a un estado cuántico dentro del espacio de Hilbert. Para clasificar correctamente necesitamos no linealidad, y como la evolución cuántica es lineal en el espacio de Hilbert, la no linealidad respecto a la entrada viene de cómo codificamos los datos (y de la medida); por eso la codificación es tan importante.

El bucle híbrido cuántico-clásico



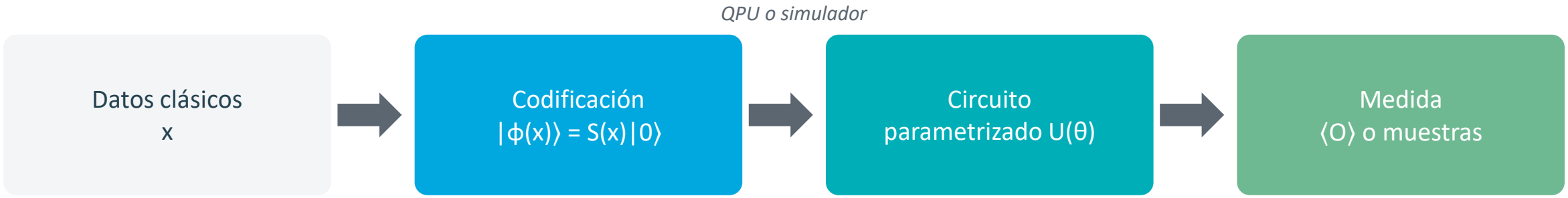
Paso 2, el circuito parametrizado U(θ), también llamado *ansatz*: capas de puertas de rotación cuyos ángulos son los parámetros entrenables, más puertas de entrelazamiento. Su arquitectura es una decisión de diseño y debe adaptarse a cada problema, como la arquitectura de una red neuronal.

El bucle híbrido cuántico-clásico



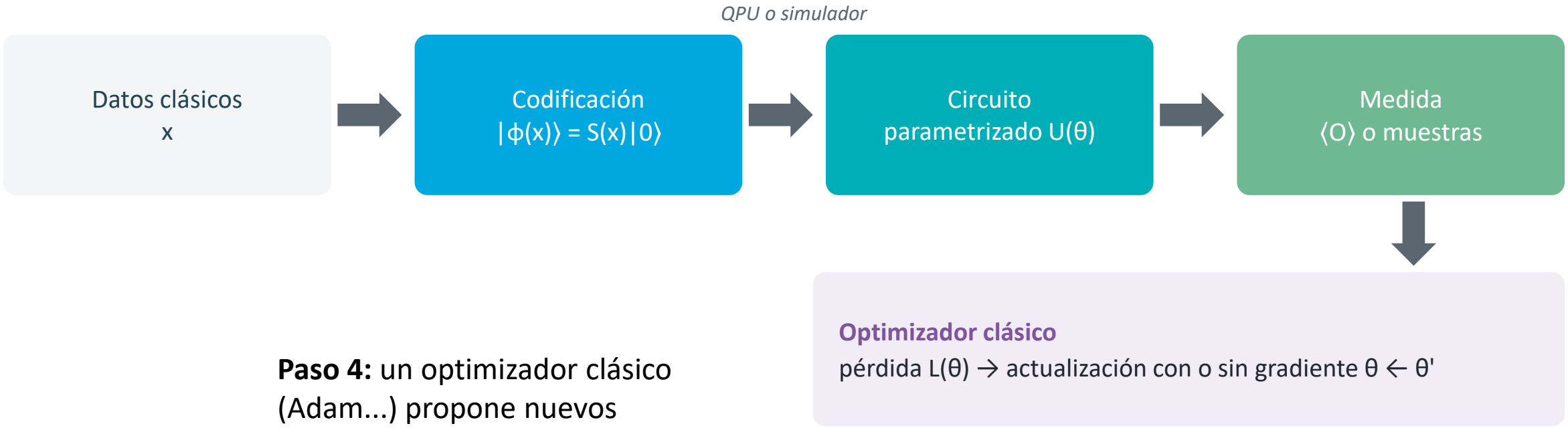
Paso 2, el circuito parametrizado U(θ), también llamado *ansatz*: *En otras palabras*, un ansatz es la estructura fija del circuito entrenable U(θ): qué puertas lleva y en qué orden. Sus ángulos θ son lo que se aprende, igual que la arquitectura de una red neuronal frente a sus pesos.

El bucle híbrido cuántico-clásico



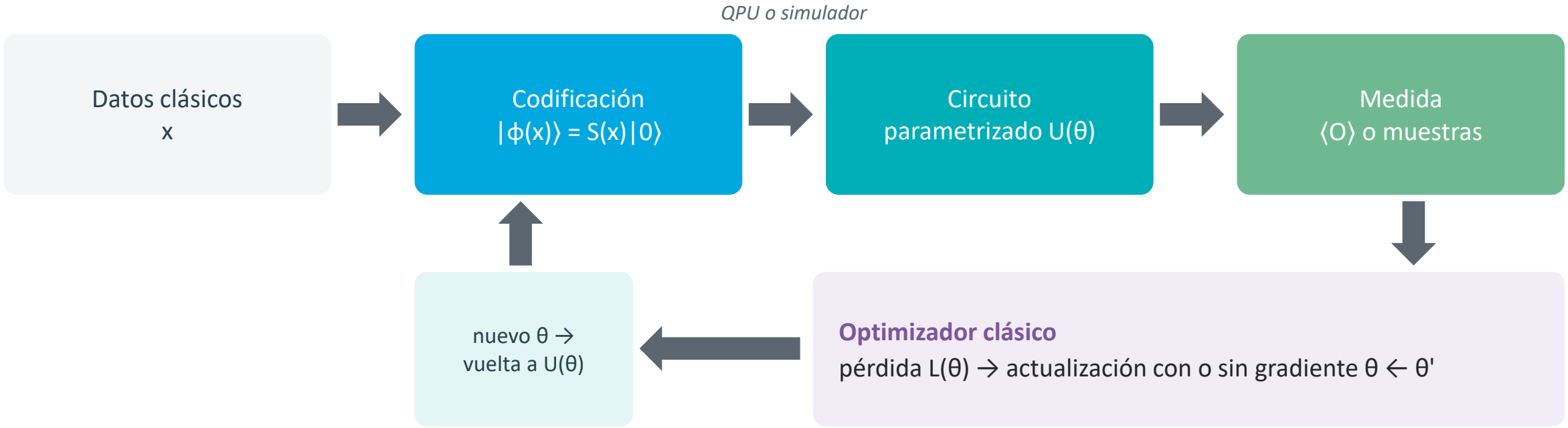
Paso 3, medida: ejecutamos el circuito muchas veces (*shots*) y estimamos un valor esperado o una distribución de probabilidad; con ello calculamos una función de coste.

El bucle híbrido cuántico-clásico



Paso 4: un optimizador clásico (Adam...) propone nuevos parámetros y el bucle se repite.

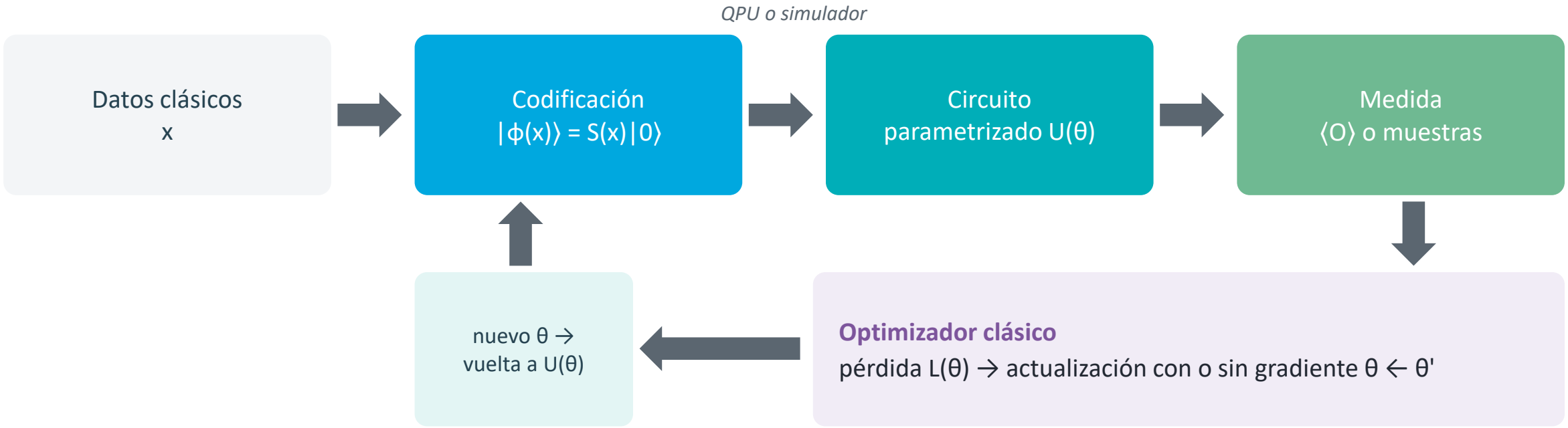
El bucle híbrido cuántico-clásico



La misma idea que entrenar una red neuronal: *forward* en el dispositivo cuántico; pérdida y actualización en CPU/GPU. Cada evaluación de la pérdida requiere muchas ejecuciones repetidas del circuito (*shots*).

Refs.: [5] Cerezo et al. (2021); [6] Benedetti et al. (2019); [13] Mitarai et al. (2018); [28] Macaluso et al. (2020).

El bucle híbrido cuántico-clásico



Nota: como cada evaluación de la pérdida necesita muchos shots, importan el coste y la latencia entre CPU y QPU.

Refs.: [5] Cerezo et al. (2021); [6] Benedetti et al. (2019); [13] Mitarai et al. (2018); [28] Macaluso et al. (2020).

Feature encoding: llevar datos clásicos a qubits

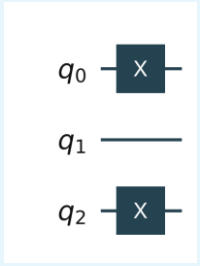
Ideas clave:

- Codificar datos clásicos en estados cuánticos.
- Es una parte muy delicada del procesamiento de la información cuántica porque define el mapa que proyectará los datos clásicos en el espacio de Hilbert.
- Hay múltiples técnicas.

Feature encoding: llevar datos clásicos a qubits

Basis encoding

Cadena de bits → estado de la base computacional. 101 → |101⟩



Coste:
n bits → n qubits

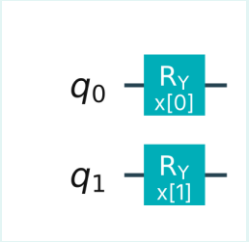
Basis encoding asocia cada elemento del dataset a un estado de la base computacional; por eso los datos deben representarse como una cadena de bits de longitud igual al número de qubits, lo que da una correspondencia uno a uno entre elementos del dataset y vectores de la base.

Refs.: [2] Schuld y Petruccione (2021); [11] LaRose y Coyle, PRA (2020). Circuitos dibujados con Qiskit 2.6.

Feature encoding: llevar datos clásicos a qubits

Angle encoding

Cada característica es un ángulo de rotación. $x_i \rightarrow RY(x_i)$ en el qubit i



Coste:
n características → n qubits, profundidad 1

Angle encoding usa cada característica como ángulo de rotación: un qubit por característica y un circuito de profundidad constante, por eso es la opción más común en dispositivos NISQ. Normalmente se reescalan las características, por ejemplo a [0, π], porque las rotaciones son periódicas.

Refs.: [2] Schuld y Petruccione (2021); [11] LaRose y Coyle, PRA (2020). Circuitos dibujados con Qiskit 2.6.

Feature encoding: llevar datos clásicos a qubits

Amplitude encoding es muy compacto: 2 elevado a n valores en n qubits. Pero preparar un estado arbitrario requiere en general un número de puertas que crece exponencialmente con n, lo que puede anular cualquier aceleración.

Amplitude encoding

Vector normalizado almacenado en las amplitudes: $\Sigma\, x_i\, |i\rangle$

$$|\psi(x)\rangle = \Sigma_i\, x_i\, |i\rangle \,/\, \|x\|$$

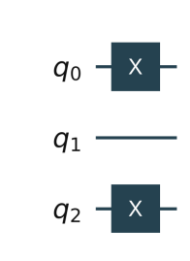
Coste:
2^n características → n qubits, pero la preparación genérica necesita ~2^n puertas

Refs.: [2] Schuld y Petruccione (2021); [11] LaRose y Coyle, PRA (2020). Circuitos dibujados con Qiskit 2.6.

Feature encoding: llevar datos clásicos a qubits

Basis encoding

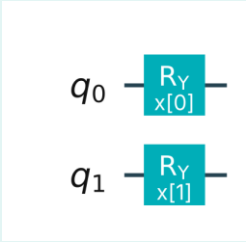
Cadena de bits → estado de la base computacional. 101 → |101⟩



Coste:
n bits → n qubits

Angle encoding

Cada característica es un ángulo de rotación. x_i → RY(x_i) en el qubit i



Coste:
n características → n qubits, profundidad 1

Amplitude encoding

Vector normalizado almacenado en las amplitudes: Σ x_i |i⟩

$$|\psi(x)\rangle = \Sigma_i \, x_i \, |i\rangle \, / \, \|x\|$$

Coste:
2^n características → n qubits, pero la preparación genérica necesita ~2^n puertas

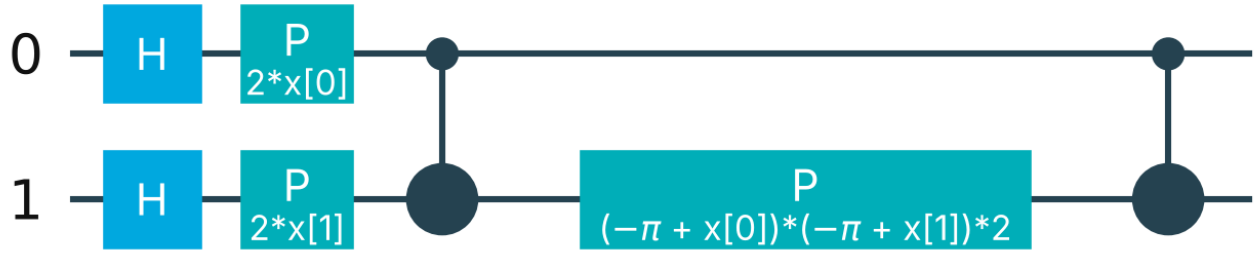
Refs.: [2] Schuld y Petruccione (2021); [11] LaRose y Coyle, PRA (2020). Circuitos dibujados con Qiskit 2.6.

Feature encoding: llevar datos clásicos a qubits

Hay más:

- **Mapas tipo IQP o ZZ** (Havlíček et al., 2019): rotaciones de fase con los datos más términos de interacción entre qubits que los entrelazan. Es el que usamos en los laboratorios.
- **Dense angle encoding** (LaRose y Coyle, 2020): mete 2 características por qubit usando dos ángulos de rotación en lugar de uno.
- **Data re-uploading** (Pérez-Salinas et al., 2020): carga el mismo dato varias veces, intercalado con capas entrenables.
- **Hamiltonian encoding** (Schuld y Petruccione): el dato fija los parámetros de una evolución temporal del sistema.
- **Embeddings entrenables** (Lloyd et al., 2020, arXiv:2001.03622): la propia codificación tiene parámetros que se optimizan para separar mejor las clases.

Un feature map muy utilizado: ZZ (tipo IQP)



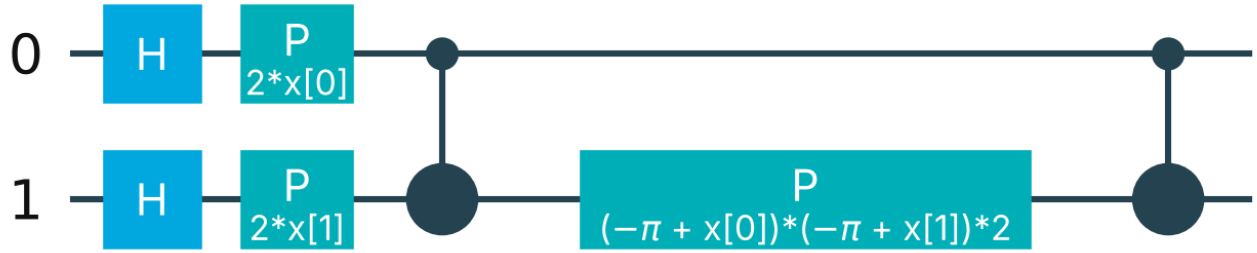
- **H** en cada qubit y después rotaciones de fase $P(2\cdot\phi(x))$.
- Los términos por pares $\phi(x_i, x_j) = (\pi - x_i)(\pi - x_j)$ entre CNOT generan un entrelazamiento que depende del dato.
- Se repite `reps` veces (2 por defecto en Qiskit).

Por qué es popular

- Lo propusieron Havlíček et al. (Nature, 2019) –aríticulo de IBM- junto con el clasificador de kernel cuántico.
- Se conjetura difícil de simular clásicamente a medida que crece el circuito (familia IQP).
- Disponible como zz_feature_map en Qiskit; fácil de escribir a mano en PennyLane (Lab. 2).

Refs.: [7] Havlíček et al., Nature 567, 209 (2019); documentación de zz_feature_map en Qiskit (φ por defecto).

Un feature map muy utilizado: ZZ (tipo IQP)



- **H** en cada qubit y después rotaciones de fase $P(2\cdot\phi(x))$.
- Los términos por pares $\phi(x_i, x_j) = (\pi - x_i)(\pi - x_j)$ entre CNOT generan un entrelazamiento que depende del dato.
- Se repite `reps` veces (2 por defecto en Qiskit).

Aclaraciones

- El feature map es el circuito que convierte un dato clásico (un vector de números) en un estado cuántico, usando esos números como ángulos de rotación de las puertas. Es el paso de «cargar los datos en los qubits» y decide qué similitudes y qué funciones puede captar el modelo, por eso se trata como un hiperparámetro.

Refs.: [7] Havlíček et al., Nature 567, 209 (2019); documentación de zz_feature_map en Qiskit (φ por defecto).

Cómo escoger el feature encoding

1 Recursos

Qubits y profundidad de circuito que puedes permitirte en el dispositivo o simulador.

2 Expresividad

Con angle encoding el modelo es una serie de Fourier en x; repetir la codificación (re-uploading) añade frecuencias.

3 Entrenabilidad

Mapas muy expresivos o muy entrelazantes pueden concentrar los kernels y anular los gradientes.

4 Robustez

Algunas codificaciones se degradan más que otras con el ruido del hardware.

Regla práctica: probad varias codificaciones y comparad siempre con un modelo clásico..

Refs.: [10] Schuld, Sweke y Meyer, PRA (2021); [12] Pérez-Salinas et al., Quantum (2020); [17] Thanasilp et al. (2024); [11] LaRose y Coyle (2020).

Dos familias de modelos: kernels vs. variacionales

Kernel cuántico (QSVC / QSVR)

- El feature map lleva cada punto a un estado cuántico y el kernel es el solapamiento al cuadrado entre dos estados.
- La optimización de la SVM sigue siendo clásica.
- Coste: $O(N^2)$ evaluaciones del kernel para N datos de entrenamiento. Hace falta un circuito por cada par de puntos, así que escala cuadráticamente con el tamaño del conjunto de entrenamiento

Modelo variacional (VQC / VQR / QNN)

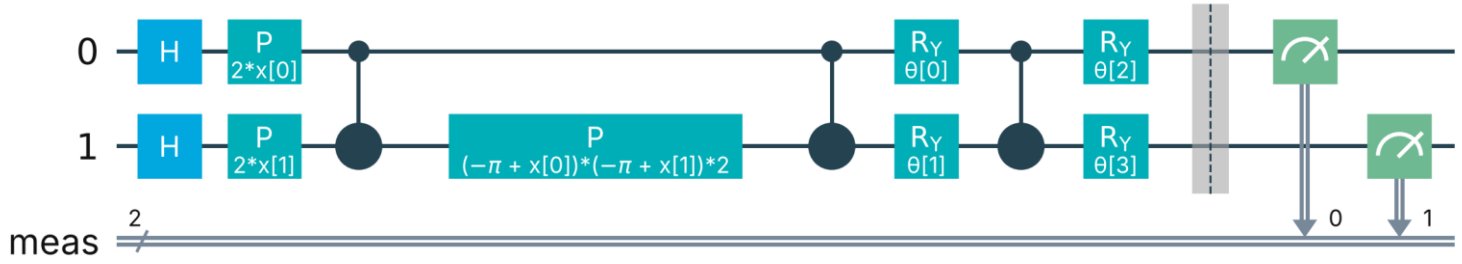
- Tras la codificación aplicamos un circuito entrenable $U(\theta)$ y medimos un observable. Los parámetros se optimizan de forma iterativa. θ se entrena con un optimizador clásico, como una red neuronal.
- Coste: shots × iteraciones.

En hardware: se ejecuta $U(x)$ y después $U^{\dagger}(x')$ y se mide; la probabilidad de de volver al estado todo ceros.

Relación entre ambos: los modelos variacionales supervisados también pueden reescribirse como métodos de kernel (Schuld, 2021).

Refs.: [7] Havlíček et al. (2019); [8] Schuld y Killoran, PRL (2019); [9] Schuld (2021).

Entrenar un Variational Quantum Classifier (VQC)



Feature map ZZ (reps=1) → ansatz real_amplitudes (reps=1) → medida

Salida → clase

- Cada cadena de bits medida se asigna a una clase (x mod nº de clases) → probabilidades de clase.
- Pérdida: entropía cruzada.

¿Qué es?

Algoritmo de aprendizaje supervisado que usa un circuito cuántico variacional, también llamado circuito cuántico parametrizado, con una estructura parecida a la de una red neuronal clásica: cada puerta parametrizada equivale a un nodo y los parámetros que la gobiernan a los pesos.

¿Cómo funciona?

Tiene una fase de entrenamiento con datos etiquetados y una fase de validación en la que se clasifican datos nuevos para evaluar la calidad del modelo. Es un algoritmo híbrido: combina una componente cuántica y una clásica que optimiza los parámetros del circuito en bucle.

Refs.: [7] Havlíček et al. (2019); [13] Mitarai et al. (2018); [14] Schuld et al., PRA (2019); [26] doc. de Qiskit ML.

Baño de realidad: retos abiertos

Barren plateaus

Los gradientes se anulan exponencialmente con el número de qubits en muchos circuitos aleatorios o profundos.

[15][16]

Concentración del kernel

Los valores del kernel son casi idénticos para todas las entradas: hacen falta exponencialmente muchos shots.

[17]

Carga de datos

Codificar grandes datasets clásicos puede costar más que el propio cálculo.

[2]

Descuantización

Algunas aceleraciones anunciadas tienen algoritmos clásicos «inspirados en cuántica» con escalado similar.

[19]

Benchmarks

En 160 datasets pequeños, los modelos clásicos estándar superaron en general a 12 modelos cuánticos.

[20]

Ruido

Los errores NISQ limitan la profundidad; la mitigación de errores añade sobrecoste de muestreo.

[25]

Refs.: [15] McClean et al. (2018); [16] Larocca et al. (2025); [17] Thanasilp et al. (2024); [19] Tang (2019); [20] Bowles, Ahmed y Schuld (2024); [25] Preskill (2018).

Casos de uso: clasificación, clustering, regresión

Tarea	Enfoque cuántico (NISQ)	Qiskit Machine Learning	PennyLane	Lab.
Clasificación	Kernel cuántico + SVM; clasificador variacional	QSVC, PegasosQSVC, VQC	QNode + SVC (precalculado) / VQC propio	1, 2
Clustering	Kernel cuántico como similitud para un algoritmo clásico	FidelityQuantumKernel → scikit-learn	Matriz de kernel → SpectralClustering	2
Regresión	Regresor variacional; regresión con kernel	VQR, QSVR	QNode con re-uploading + Adam	2

También hay propuestas tolerantes a fallos (QSVM, q-means), pero suponen RAM cuántica y hardware con corrección de errores.

Refs.: [26] Sahin et al., Qiskit Machine Learning (2025); [27] Bergholm et al., PennyLane (2018); [22] Lloyd et al. (2013); [23] Kerenidis et al. (2019); [24] Rebentrost et al. (2014).

Caso de uso 1 – Clasificación

Cómo funciona

- Se codifica x con un feature map (p. ej. ZZ).
- Patrones:
  - Opción A (QSVC): se calcula la matriz del kernel cuántico y se entrena una SVM clásica.
  - Opción B (VQC): se añade un *ansatz* entrenable, se mide, se asignan resultados a probabilidades de clase y se minimiza la entropía cruzada.
  - Opción C (re-uploading): incluso un único qubit puede clasificar si el dato se codifica varias veces.

Qiskit – misma API que scikit-learn

```
from qiskit_machine_learning.algorithms import QSVC
from qiskit_machine_learning.kernels import \
    FidelityStatevectorKernel

kernel = FidelityStatevectorKernel(
    feature_map=zz_feature_map(2, reps=2))
qsvc = QSVC(quantum_kernel=kernel)
qsvc.fit(X_train, y_train)
qsvc.score(X_test, y_test)
```

Demostrado en hardware: Havlíček et al. (2019) ejecutaron un clasificador variacional y uno de kernel en 2 qubits superconductores, con datos artificiales diseñados para el feature map.

Refs.: [7] Havlíček et al. (2019); [12] Pérez-Salinas et al. (2020); [26] Qiskit ML (2025).

Caso de uso 1 – Clasificación

Cómo funciona

- FidelityStatevectorKernel calcula el solapamiento exacto a partir de vectores de estado, rápido para docencia.
- En hardware se usaría FidelityQuantumKernel con un sampler, que estima los solapamientos con shots.

Qiskit – misma API que scikit-learn

```
from qiskit_machine_learning.algorithms import QSVC
from qiskit_machine_learning.kernels import \
    FidelityStatevectorKernel

kernel = FidelityStatevectorKernel(
    feature_map=zz_feature_map(2, reps=2))
qsvc = QSVC(quantum_kernel=kernel)
qsvc.fit(X_train, y_train)
qsvc.score(X_test, y_test)
```

Demostrado en hardware: Havlíček et al. (2019) ejecutaron un clasificador variacional y uno de kernel en 2 qubits superconductores, con datos artificiales diseñados para el feature map.

Refs.: [7] Havlíček et al. (2019); [12] Pérez-Salinas et al. (2020); [26] Qiskit ML (2025).

Caso de uso 2 – Clustering

1 Codificar

cada punto con el feature map

2 Matriz de kernel

$K[i, j] = |\langle \phi(x_j) | \phi(x_i) \rangle|^2$ para todos los pares

3 Algoritmo clásico

clustering espectral o kernel k-means sobre K

4 Evaluar

índice de Rand ajustado (ARI) frente a las etiquetas reales, si se conocen

Era NISQ

La forma más sencilla de usar un ordenador cuántico es, de nuevo, a través de un kernel.

Conclusión

Cualquier algoritmo de clustering basado en kernels puede usar un kernel cuántico sin cambios.

Su calidad depende de que el feature map encaje con la geometría de los datos.

Refs.: [8] Schuld y Killoran (2019); [22] Lloyd, Mohseni y Rebentrost (2013); [23] Kerenidis et al. (2019). Resultados: cuaderno del Lab. 2, semilla 42.

Caso de uso 2 – Clustering

1 Codificar

cada punto con el feature map

2 Matriz de kernel

$K[i, j] = |\langle \phi(x_j) | \phi(x_i) \rangle|^2$ para todos los pares

3 Algoritmo clásico

clustering espectral o kernel k-means sobre K

4 Evaluar

índice de Rand ajustado (ARI) frente a las etiquetas reales, si se conocen

Resultado del Lab. 2 (Two moons, 100 puntos)

Clustering espectral, kernel RBF (clásico): ARI = **0,54**

Clustering espectral, kernel cuántico ZZ: ARI = **0,46**

ARI: 1 = coincidencia perfecta, ≈0 = azar.

Mide si el clustering ha juntado los mismos puntos que estaban juntos en la realidad

Conclusión

Cualquier algoritmo de clustering basado en kernels puede usar un kernel cuántico sin cambios.

Su calidad depende de que el feature map encaje con la geometría de los datos.

Refs.: [8] Schuld y Killoran (2019); [22] Lloyd, Mohseni y Rebentrost (2013); [23] Kerenidis et al. (2019). Resultados: cuaderno del Lab. 2, semilla 42.

ML vs. QML: mismo pipeline, distinto modelo

Dataset Iris
- 150 flores
- 4 características
- 3 clases

ML clásico (scikit-learn)

```
from sklearn.svm import SVC


model = SVC(kernel="rbf")


model.fit(X_train, y_train)
model.score(X_test, y_test)
```

ML cuántico (Qiskit Machine Learning)

```
from qiskit_machine_learning.algorithms import QSVC
from qiskit_machine_learning.kernels import \
    FidelityStatevectorKernel
fmap = zz_feature_map(2, reps=2)
model = QSVC(quantum_kernel=
    FidelityStatevectorKernel(feature_map=fmap))
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

Pasos comunes: Iris → estandarizar → PCA a 2 características → escalar a [0, π] porque los ángulos de rotación son periódicos→ partición estratificada 80/20 (semilla 42). Solo cambia la línea del modelo.

Refs.: [26] Sahin et al. (2025); [29] Pedregosa et al. (2011). Código: cuaderno Lab1_Qiskit_Clasificacion.ipynb.

ML vs. QML: mismo pipeline, distinto modelo

Dataset Iris
- 150 flores
- 4 características
- 3 clases

ML clásico (scikit-learn)

```
from sklearn.svm import SVC

model = SVC(kernel="rbf")

model.fit(X_train, y_train)
model.score(X_test, y_test)
```

ML cuántico (Qiskit Machine Learning)

```
from qiskit_machine_learning.algorithms import QSVC
from qiskit_machine_learning.kernels import \
    FidelityStatevectorKernel
fmap = zz_feature_map(2, reps=2)
model = QSVC(quantum_kernel=
    FidelityStatevectorKernel(feature_map=fmap))
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

Pasos comunes: Iris → estandarizar → PCA a 2 características → escalar a [0, π] porque los ángulos de rotación son periódicos→ partición estratificada 80/20 (semilla 42). Solo cambia la línea del modelo.

Refs.: [26] Sahin et al. (2025); [29] Pedregosa et al. (2011). Código: cuaderno Lab1_Qiskit_Clasificacion.ipynb.

ML vs. QML: mismo pipeline, distinto modelo

Dataset Iris
- 150 flores
- 4 características
- 3 clases

ML clásico (scikit-learn)

```
from sklearn.svm import SVC

model = SVC(kernel="rbf")

model.fit(X_train, y_train)
model.score(X_test, y_test)
```

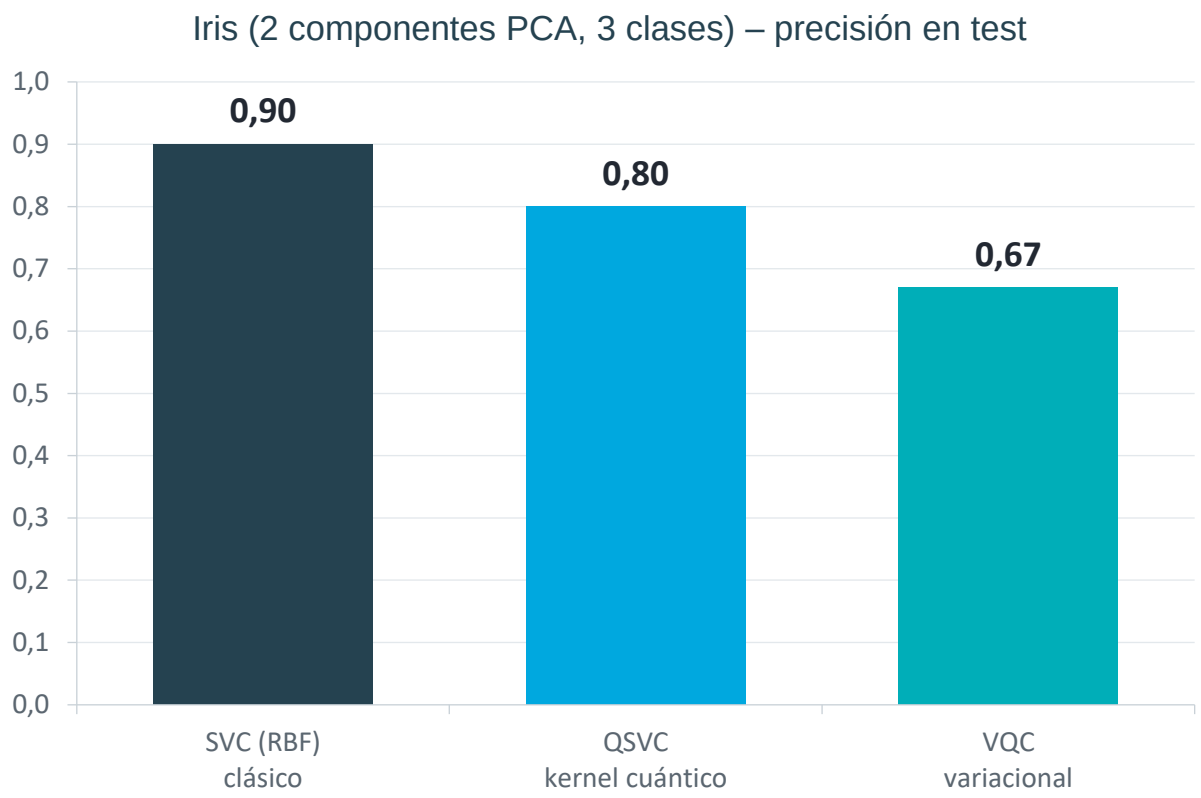
ML cuántico (Qiskit Machine Learning)

```
from qiskit_machine_learning.algorithms import QSVC
from qiskit_machine_learning.kernels import \
    FidelityStatevectorKernel
fmap = zz_feature_map(2, reps=2)
model = QSVC(quantum_kernel=
    FidelityStatevectorKernel(feature_map=fmap))
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

NOTA: QSVC hereda de la SVC de scikit-learn, así que fit, predict y score se comportan igual. Esto es relevante para un perfil informático: se puede insertar un modelo cuántico en un pipeline existente, una búsqueda de hiperparámetros o una validación cruzada sin reescribir el resto.

Refs.: [26] Sahin et al. (2025); [29] Pedregosa et al. (2011). Código: cuaderno Lab1_Qiskit_Clasificacion.ipynb.

ML vs. QML: resultados sobre los mismos datos



Datos: cuadernos de los Lab. 1 y 2, simuladores sin ruido, semilla 42, 30 muestras de test (Iris). No es un benchmark en hardware.

Two moons (Lab. 2, PennyLane)

- SVC con kernel RBF (clásico): precisión en test 0,93
- SVC con kernel cuántico ZZ: precisión en test 0,77

Cómo leer los resultados

- Datos pequeños, clásicos y de baja dimensión: el modelo clásico gana o empata.
- Coherente con estudios de benchmark amplios [20].

Entonces, ¿cuándo puede tener sentido el QML?

1 Datos cuánticos (QQ)

Datos producidos por sensores, simulaciones o dispositivos cuánticos; si el dato ya vive en un sistema cuántico, desaparece el cuello de botella de la codificación.

2 Separaciones demostrables

Problemas construidos a propósito (p. ej. basados en el logaritmo discreto) con aceleración cuántica demostrada para aprender.

3 Investigación y herramientas

Estudiar entrenabilidad, codificaciones y benchmarks; preparar flujos de trabajo híbridos HPC–cuántica.

4 Primero, un test con datos

Usar criterios como la diferencia geométrica entre kernels antes de esperar una ventaja.

Postura de ingeniería: prototipar en simuladores, comparar con referencias clásicas sólidas y después probar en hardware.

Refs.: [18] Huang et al., Nat. Commun. (2021); [21] Liu, Arunachalam y Temme, Nat. Phys. (2021); [20] Bowles et al. (2024).

Práctica – preparación

Instalación (Python 3.11 – 3.13)

```
python -m venv qml && source qml/bin/activate
pip install -r requirements.txt
pip check   # debe decir: No broken requirements
jupyter lab # abre los cuadernos Lab1 y Lab2
```

1. Lab1_Qiskit_Clasificacion.ipynb
2. Lab2_PennyLane_…_Regresion.ipynb

Paquete	Versión probada
Python	3.11 · 3.12 · 3.13
qiskit	2.6.1
qiskit-machine-learning	0.9.1
pennylane	0.45.1
scikit-learn	1.9.1
jupyterlab	4.6.4

Funciona en la CPU de un portátil. Cálculo con «Run All»: Lab. 1 ≈ 35 s, Lab. 2 ≈ 90 s (equipo de preparación). Sin cuenta en hardware cuántico.

Versiones usadas para generar todos los resultados de esta presentación (octubre de 2026). Versiones más nuevas pueden cambiar la API.

Lab. 1 – Qiskit: clasificadores clásicos vs. cuánticos

1. **Secc. 1 · Datos: Iris en 2 qubits**

   `StandardScaler → PCA(2) → MinMaxScaler(0, π) → partición 80/20`

2. **Secc. 2 · Referencia clásica**

   `SVC(kernel="rbf")`

3. **Secc. 3–4 · Feature map y kernel**

   `zz_feature_map(2, reps=2) + FidelityStatevectorKernel + QSVC`

4. **Secc. 5 · Clasificador variacional**

   `VQC(..., COBYLA(maxiter=100), output_shape=3) + curva de pérdida`

5. **Secc. 6 · Comparar**

   `precisión en test y matrices de confusión`

Explicación

- Sección 1, preprocesado: fijaos en el reescalado a [0, π]; preguntad por qué (los ángulos de rotación son periódicos). La gráfica muestra que setosa se separa bien y versicolor y virginica se solapan.
- Sección 2, referencia clásica, que tarda milisegundos.

Cuaderno: Lab1_Qiskit_Clasificacion.ipynb (también como script .py). Se omiten los tiempos: dependen del equipo.

Lab. 1 – Qiskit: clasificadores clásicos vs. cuánticos

1. **Secc. 1 · Datos: Iris en 2 qubits**

   `StandardScaler → PCA(2) → MinMaxScaler(0, π) → partición 80/20`

2. **Secc. 2 · Referencia clásica**

   `SVC(kernel="rbf")`

3. **Secc. 3–4 · Feature map y kernel**

   `zz_feature_map(2, reps=2) + FidelityStatevectorKernel + QSVC`

4. **Secc. 5 · Clasificador variacional**

   `VQC(..., COBYLA(maxiter=100), output_shape=3) + curva de pérdida`

5. **Secc. 6 · Comparar**

   `precisión en test y matrices de confusión`

Explicación

- Secciones 3 y 4: se dibuja el feature map ZZ y se calcula el kernel exacto con FidelityStatevectorKernel. El mapa de calor y la tabla de medias por bloque dan un buen punto de debate: el kernel es mayor dentro de cada especie, pero setosa y virginica salen casi tan parecidas (0,31) como setosa consigo misma (0,34), aunque en la gráfica están en extremos opuestos: codificar los datos como ángulos puede acercar puntos lejanos. La celda opcional 4.1 (PROBAR_SHOTS = True) estima el mismo kernel con shots, como haría el hardware; tarda alrededor de un minuto en lugar de menos de un segundo y la precisión de test no cambia.

Cuaderno: Lab1_Qiskit_Clasificacion.ipynb (también como script .py). Se omiten los tiempos: dependen del equipo.

Lab. 1 – Qiskit: clasificadores clásicos vs. cuánticos

1. **Secc. 1 · Datos: Iris en 2 qubits**

   `StandardScaler → PCA(2) → MinMaxScaler(0, π) → partición 80/20`

2. **Secc. 2 · Referencia clásica**

   `SVC(kernel="rbf")`

3. **Secc. 3–4 · Feature map y kernel**

   `zz_feature_map(2, reps=2) + FidelityStatevectorKernel + QSVC`

4. **Secc. 5 · Clasificador variacional**

   `VQC(..., COBYLA(maxiter=100), output_shape=3) + curva de pérdida`

5. **Secc. 6 · Comparar**

   `precisión en test y matrices de confusión`

Explicación

- Sección 5, clasificador variacional con feature map ZZ, ansatz real_amplitudes con 3 repeticiones y COBYLA limitado a 100 iteraciones, y output_shape=3 porque hay 3 clases (sin ese parámetro Qiskit usa 2 clases y el modelo no puede predecir la tercera; es un error fácil de cometer y conviene comentarlo). Es el modelo más lento y, aquí, el menos preciso. La curva de pérdida muestra que COBYLA se detiene en 86 evaluaciones, antes del máximo de 100.

Cuaderno: Lab1_Qiskit_Clasificacion.ipynb (también como script .py). Se omiten los tiempos: dependen del equipo.

Lab. 1 – Qiskit: clasificadores clásicos vs. cuánticos

1. **Secc. 1 · Datos: Iris en 2 qubits**

   `StandardScaler → PCA(2) → MinMaxScaler(0, π) → partición 80/20`

2. **Secc. 2 · Referencia clásica**

   `SVC(kernel="rbf")`

3. **Secc. 3–4 · Feature map y kernel**

   `zz_feature_map(2, reps=2) + FidelityStatevectorKernel + QSVC`

4. **Secc. 5 · Clasificador variacional**

   `VQC(..., COBYLA(maxiter=100), output_shape=3) + curva de pérdida`

5. **Secc. 6 · Comparar**

   `precisión en test y matrices de confusión`

Explicación

- Sección 6: comparación final con matrices de confusión; la virginica y la versicolor son las que se confunden. A la derecha está la salida esperada con semilla 42. Animad al alumnado a cambiar una sola cosa cada vez y anotar el efecto, como en un registro de experimentos de ML.

Cuaderno: Lab1_Qiskit_Clasificacion.ipynb (también como script .py). Se omiten los tiempos: dependen del equipo.

Lab. 1 – Qiskit: clasificadores clásicos vs. cuánticos

1. **Secc. 1 · Datos: Iris en 2 qubits**

   `StandardScaler → PCA(2) → MinMaxScaler(0, π) → partición 80/20`

2. **Secc. 2 · Referencia clásica**

   `SVC(kernel="rbf")`

3. **Secc. 3–4 · Feature map y kernel**

   `zz_feature_map(2, reps=2) + FidelityStatevectorKernel + QSVC`

4. **Secc. 5 · Clasificador variacional**

   `VQC(..., COBYLA(maxiter=100), output_shape=3) + curva de pérdida`

5. **Secc. 6 · Comparar**

   `precisión en test y matrices de confusión`

Salida esperada (semilla 42)

```
# Secc. 6 · Comparación final
SVC (RBF)   entren.=0.92 test=0.90
QSVC        entren.=0.79 test=0.80
VQC         entren.=0.65 test=0.67
```

Prueba

- reps=1 y reps=3 en el feature map (Secc. 3)
- COBYLA(maxiter=300) en el VQC (Secc. 5)
- PROBAR_SHOTS = True: kernel con shots (Secc. 4.1)

Cuaderno: Lab1_Qiskit_Clasificacion.ipynb (también como script .py). Se omiten los tiempos: dependen del equipo.

Lab. 2 – PennyLane: kernels, clustering y regresión

El kernel, escrito a mano

```
def feature_map(x):          # = zz_feature_map de Qiskit
    for i in range(2):
        qml.Hadamard(wires=i)
        qml.RZ(2 * x[i], wires=i)
    qml.CNOT(wires=[0, 1])
    qml.RZ(2*(np.pi-x[0])*(np.pi-x[1]), wires=1)
    qml.CNOT(wires=[0, 1])

@qml.qnode(dev)
def circuito_kernel(x1, x2):
    feature_map(x1)
    qml.adjoint(feature_map)(x2)
    return qml.probs(wires=[0, 1])   # k = p(|00⟩)
```

Explicación

El Lab 2 muestra los mismos conceptos con menos abstracción. El feature map se escribe puerta a puerta: es el mismo mapa ZZ que zz_feature_map de Qiskit con una repetición; RZ y la puerta de fase P solo difieren en una fase global, que no afecta a las probabilidades. El circuito del kernel aplica el feature map para x1 y después el adjunto (la inversa) para x2; la probabilidad de medir 00 es el solapamiento al cuadrado, es decir, el kernel cuántico.

Idea clave: una misma matriz de kernel cuántico alimenta a SVC (supervisado) y a SpectralClustering (no supervisado); el regresor se entrena con diferenciación automática + Adam.

Cuaderno: Lab2_PennyLane_Kernel_Clustering_Regresion.ipynb (también como script .py). Refs.: [27] PennyLane; [7] Havlíček et al.

Lab. 2 – PennyLane: kernels, clustering y regresión

El kernel, escrito a mano

```
def feature_map(x):          # = zz_feature_map de Qiskit
    for i in range(2):
        qml.Hadamard(wires=i)
        qml.RZ(2 * x[i], wires=i)
    qml.CNOT(wires=[0, 1])
    qml.RZ(2*(np.pi-x[0])*(np.pi-x[1]), wires=1)
    qml.CNOT(wires=[0, 1])

@qml.qnode(dev)
def circuito_kernel(x1, x2):
    feature_map(x1)
    qml.adjoint(feature_map)(x2)
    return qml.probs(wires=[0, 1])   # k = p(|00⟩)
```

Explicación

Con esa única función hacemos tres tareas: clasificación con SVC usando un kernel precalculado, clustering con SpectralClustering usando el kernel como afinidad y, por separado, regresión con un modelo de un qubit con data re-uploading entrenado con la diferenciación automática de Penny y Adam.

Idea clave: una misma matriz de kernel cuántico alimenta a SVC (supervisado) y a SpectralClustering (no supervisado); el regresor se entrena con diferenciación automática + Adam.

Cuaderno: Lab2_PennyLane_Kernel_Clustering_Regresion.ipynb (también como script .py). Refs.: [27] PennyLane; [7] Havlíček et al.

Lab. 2 – PennyLane: kernels, clustering y regresión

El kernel, escrito a mano

```
def feature_map(x):          # = zz_feature_map de Qiskit
    for i in range(2):
        qml.Hadamard(wires=i)
        qml.RZ(2 * x[i], wires=i)
    qml.CNOT(wires=[0, 1])
    qml.RZ(2*(np.pi-x[0])*(np.pi-x[1]), wires=1)
    qml.CNOT(wires=[0, 1])

@qml.qnode(dev)
def circuito_kernel(x1, x2):
    feature_map(x1)
    qml.adjoint(feature_map)(x2)
    return qml.probs(wires=[0, 1])   # k = p(|00⟩)
```

Salida esperada (semilla 42)

```
== Clasificación (dos lunas) ==
SVC RBF            test=0.93
SVC kernel cuánt. test=0.77
== Clustering (dos lunas) ==
Espectral RBF            ARI=0.54
Espectral kernel cuánt. ARI=0.46
== Regresión sin(x) ==
VQR  MSE test=0.0002  parámetros=9
SVR RBF  MSE test=0.0055
```

Idea clave: una misma matriz de kernel cuántico alimenta a SVC (supervisado) y a SpectralClustering (no supervisado); el regresor se entrena con diferenciación automática + Adam.

Cuaderno: Lab2_PennyLane_Kernel_Clustering_Regresion.ipynb (también como script .py). Refs.: [27] PennyLane; [7] Havlíček et al.

Retos finales

1 Codificación (Lab. 1, Secc. 3)

Prueba reps = 1 y reps = 3 en el feature map. ¿Cómo cambian la precisión y el kernel?

2 Entrelazamiento (Lab. 2, Secc. 1)

Comenta las CNOT y la RZ de la interacción ZZ. ¿Mejor o peor? Compara con [20].

3 Shots (Lab. 1, Secc. 4.1)

Pon PROBAR_SHOTS = True. ¿Cuánto más lento es? ¿Cambia la precisión?

4 Sesgo inductivo (Lab. 2, Secc. 5)

Pon FREC = 5. ¿Por qué falla con CAPAS = 3? Prueba CAPAS = 5.

Tras cada cambio, vuelve a ejecutar desde esa sección y anota el resultado (registro de experimentos).

Refs.: [10] Schuld, Sweke y Meyer (2021); [20] Bowles, Ahmed y Schuld (2024).

Ideas clave

1	**QML hoy = híbrido**	Un circuito cuántico es un componente dentro de un pipeline de ML clásico.
2	**La codificación es la decisión de diseño**	Fija lo que el modelo puede expresar y lo entrenable que es.
3	**Dos familias**	Kernels cuánticos (entrenamiento convexo, $O(N^2)$ circuitos) y modelos variacionales (flexibles, más difíciles de entrenar).
4	**Comparar con honestidad**	En datasets clásicos pequeños, las referencias clásicas suelen ganar o empatar.
5	**Mismas herramientas**	Qiskit ML y PennyLane se integran con flujos de scikit-learn y PyTorch.

Bibliografía completa en las diapositivas siguientes y en la guía.

Bibliografía (1/2)

[1] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, S. Lloyd. Quantum machine learning. Nature 549, 195–202 (2017). doi:10.1038/nature23474

[2] M. Schuld, F. Petruccione. Machine Learning with Quantum Computers, 2.ª ed. Springer (2021).

[3] E. Aïmeur, G. Brassard, S. Gambs. Machine learning in a quantum world. Advances in Artificial Intelligence, LNCS 4013, 431–442 (2006).

[4] V. Dunjko, J. M. Taylor, H. J. Briegel. Quantum-enhanced machine learning. Phys. Rev. Lett. 117, 130501 (2016). arXiv:1610.08251

[5] M. Cerezo et al. Variational quantum algorithms. Nat. Rev. Phys. 3, 625–644 (2021). arXiv:2012.09265

[6] M. Benedetti, E. Lloyd, S. Sack, M. Fiorentini. Parameterized quantum circuits as machine learning models. Quantum Sci. Technol. 4, 043001 (2019). arXiv:1906.07682

[7] V. Havlíček et al. Supervised learning with quantum-enhanced feature spaces. Nature 567, 209–212 (2019). doi:10.1038/s41586-019-0980-2

[8] M. Schuld, N. Killoran. Quantum machine learning in feature Hilbert spaces. Phys. Rev. Lett. 122, 040504 (2019). arXiv:1803.07128

[9] M. Schuld. Supervised quantum machine learning models are kernel methods. arXiv:2101.11020 (2021).

[10] M. Schuld, R. Sweke, J. J. Meyer. Effect of data encoding on the expressive power of variational quantum machine-learning models. Phys. Rev. A 103, 032430 (2021). arXiv:2008.08605

[11] R. LaRose, B. Coyle. Robust data encodings for quantum classifiers. Phys. Rev. A 102, 032420 (2020). arXiv:2003.01695

[12] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, J. I. Latorre. Data re-uploading for a universal quantum classifier. Quantum 4, 226 (2020). arXiv:1907.02085

[13] K. Mitarai, M. Negoro, M. Kitagawa, K. Fujii. Quantum circuit learning. Phys. Rev. A 98, 032309 (2018). arXiv:1803.00745

[14] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, N. Killoran. Evaluating analytic gradients on quantum hardware. Phys. Rev. A 99, 032331 (2019). arXiv:1811.11184

[15] J. R. McClean et al. Barren plateaus in quantum neural network training landscapes. Nat. Commun. 9, 4812 (2018). arXiv:1803.11173

Bibliografía (2/2)

[16] M. Larocca et al. Barren plateaus in variational quantum computing. Nat. Rev. Phys. (2025). doi:10.1038/s42254-025-00813-9

[17] S. Thanasilp, S. Wang, M. Cerezo, Z. Holmes. Exponential concentration in quantum kernel methods. Nat. Commun. 15 (2024). doi:10.1038/s41467-024-49287-w

[18] H.-Y. Huang et al. Power of data in quantum machine learning. Nat. Commun. 12, 2631 (2021). doi:10.1038/s41467-021-22539-9

[19] E. Tang. A quantum-inspired classical algorithm for recommendation systems. Proc. STOC 2019, 217–228. arXiv:1807.04271

[20] J. Bowles, S. Ahmed, M. Schuld. Better than classical? The subtle art of benchmarking quantum machine learning models. arXiv:2403.07059 (2024).

[21] Y. Liu, S. Arunachalam, K. Temme. A rigorous and robust quantum speed-up in supervised machine learning. Nat. Phys. 17, 1013–1017 (2021). arXiv:2010.02174

[22] S. Lloyd, M. Mohseni, P. Rebentrost. Quantum algorithms for supervised and unsupervised machine learning. arXiv:1307.0411 (2013).

[23] I. Kerenidis, J. Landman, A. Luongo, A. Prakash. q-means: a quantum algorithm for unsupervised machine learning. NeurIPS 32 (2019). arXiv:1812.03584

[24] P. Rebentrost, M. Mohseni, S. Lloyd. Quantum support vector machine for big data classification. Phys. Rev. Lett. 113, 130503 (2014). arXiv:1307.0471

[25] J. Preskill. Quantum computing in the NISQ era and beyond. Quantum 2, 79 (2018). arXiv:1801.00862

[26] M. E. Sahin et al. Qiskit Machine Learning: an open-source library for quantum machine learning tasks at scale on quantum hardware and classical simulators. arXiv:2505.17756 (2025).

[27] V. Bergholm et al. PennyLane: automatic differentiation of hybrid quantum-classical computations. arXiv:1811.04968 (2018).

[28] A. Macaluso, L. Clissa, S. Lodi, C. Sartori. A variational algorithm for quantum neural networks. ICCS 2020, LNCS 12142, 591–604. Springer (2020).

[29] F. Pedregosa et al. Scikit-learn: machine learning in Python. J. Mach. Learn. Res. 12, 2825–2830 (2011).

MUCHAS GRACIAS

info@scayle.es

SCAYLE SUPERCOMPUTACIÓN CASTILLA Y LEÓN
universidad de león
NOS IMPULSA
Junta de Castilla y León
Europa impulsa nuestro crecimiento
FONDO EUROPEO DE DESARROLLO REGIONAL
UNIÓN EUROPEA
Quantum SPAIN
GOBIERNO DE ESPAÑA
MINISTERIO DE ASUNTOS ECONÓMICOS Y TRANSFORMACIÓN DIGITAL
Plan de Recuperación, Transformación y Resiliencia
Financiado por la Unión Europea
NextGenerationEU
RES
RED ESPAÑOLA DE SUPERCOMPUTACIÓN
Infraestructuras Científicas y Técnicas Singulares
GOBIERNO DE ESPAÑA
MINISTERIO DE CIENCIA E INNOVACIÓN
Unión Europea
Fondo Europeo de Desarrollo Regional
"Una manera de hacer Europa"